Лекция 04 — Погружение в функции

Содержание

1. Лямбда-выражения
    1. Мотивация
    2. Представление о лямбда-выражениях
    3. Шаблонные и обобщенные лямбды
    4. Техническое устройство лямбд
2. Полиморфизм вызываемых объектов
    1. А что такое функция?
    2. `std::function`

”— Так давайте вместо лямбды просто функтор напишем.
— Так это, ну… в смысле, да, но нет.
— А. П. Хвастунов

Лямбда-выражения

Мотивация

Мы уже достаточно давно и неплохо (мы, впрочем, скоро выясним, что это не совсем так) знакомы с таким понятием, как указатель на функцию. Главная задача, которусю он позволял нам решать — это параметризация алгоритмов; например, можно вспомнить классический пример с параметризацией сортировки компаратором:

```
void ArraySort(int arr[], bool (*cmp)(int)) {
// some hard logic
}
```

Одна из проблем такого подхода состоит в том, что это банально не очень удобно. Мы можем чуть упростить себе жизнь, заведя псевдоним для типа компаратора:

```
using ComparatorType = bool (*)(int);

void ArraySort(int arr[], ComparatorType cmp) {
// some hard logic
}
```

Такой вид сигнатуры наталкивает нас на мысль о том, что можно зайти ещё дальше и сделать `ComparatorType` шаблонным параметром:

```
template<typename ComparatorType>
void ArraySort(int arr[], ComparatorType cmp) {
// some hard logic
}
```

Мы добились того, что теперь наша функция принимает не только указатель на другую функцию, но и любой вызываемый объект, за которым, конечно, в первую очередь подразумевается функтор. Это отличный единообразный интерфейс, который ощутимо упрощает решение исходной задачи.

С его использованием, впрочем, всё ещё связаны некоторые неудобства. Во-первых, для того, чтобы использовать функтор, мы обязаны где-то его объявить, причём это «где-то» неизбежно отделено от места использования, что затрудняет чтение кода. Хотя это может не казаться веским до тех пор, пока мы, например, не хотим использовать какой-нибудь стандартный функтор в нестандартном контексте:

```
auto it = std::find_if(v.begin(), v.end(), GreaterThan(4));
```

В чем проблема такого кода?
В том, что у нас уже есть функтор `std::greater`, который решает ровно нужную здесь задачу. Тем не менее, использовать мы его здесь не можем: он принимает два аргумента, и зафиксировать один из них мы, казалось бы, не можем. Разумно ли писать целый функциональный класс для того, чтобы продублировать существующую логику? Едва ли.

На деле, конечно, модифицировать этот функтор мы всё-таки можем, если вспомним про `std::bind`. Выглядеть это будет как-то так:

```
auto it = std::find_if(v.begin(), v.end(), std::bind(std::greater<int>(),
std::placeholders::_1, 4));
```

Казалось бы, всё теперь совсем уж хорошо. Но разработчики — люди с достаточно тонким чувством высокого, из-за чего общественное мнение сочло `std::bind` ... уродливым. Это действительно главная его проблема — конструкции с ним тяжело читать и писать. Вот и выходит, что все известные нам на данный момент методы параметризации функций либо громоздкие, либо нечитабельные, либо небезопасные — словом, все какие-то не такие.

Представление о лямбда-выражениях

Такой вселенской несправедливости был положен конец вместе с приходом стандарта C++11, который ввёл в язык лямбда-выражения.

Лямбда-выражение (далее допустимо просто *лямбда*) — это неименованный функтор. Стоит отдельно упомянуть, что точно так же, как и обычный функтор, лямбда может выступать частью *замыкания* — механики объединения кода функции и контекста, в котором она была объявлена; если функторы захватывали контекст при помощи конструктора, то у лямбд для этого отведён специальный механизм, который мы совсем скоро затронем.

Практически говоря, лямбда — это не что иное, как синтаксический сахар над функциональными объектами, и мы убедимся в этом, когда чуть подробнее поговорим об их устройстве.

Синтаксис объявления лямбда-выражения в простом случае имеет следующий вид:

```
[captures] (params) attr { body }
```

Всё это объявление за исключением блока `captures` практически представляет собой сигнатуру функции со списком параметров `params`, аттрибутами `attr` и телом `body`, поэтому за исключением отдельных нюансов, которые будут затронуты далее, мы не будем особенно на этом останавливаться. Стоит лишь отметить, что возвращаемый тип по умолчанию выводится при помощи `auto` [1]; явно задать его можно, указав его как trailing type.

Блок `captures` же, наоборот, заслуживает отдельного внимания. Он используется для передачи лямбде информации о переменных текущей области видимости, превращая её объявление в замыкание. Впрочем, лямбды могут использовать некоторые переменные и без захвата, но если и только если:

> 1. Это глобальные или статические переменные:
>
> ```
> int x = 238;
>
> int main() {
>  static int y = 3104;
>  auto lambda = []() {
>  std::cout << x << ' ' << y; // OK!
>  };
> }
> ```

> 2. Это ссылки на константные выражения:

```
const int x = 238;

int main() {
    const int& y = x;
    auto lambda = []() {
        std::cout << y; // OK!
    };
}
```

3. Это константные переменные целочисленного типа, инициализированные константным выражением:

```
int main() {
    const int y = 238;
    auto lambda = []() {
        std::cout << y; // OK!
    };
}
```

4. Это `constexpr` переменные без `mutable` полей:

```
struct MyStruct {
    int x;
    // ВАЖНО: с mutable не скомпилируется
};

int main() {
    constexpr MyStruct s{238};
    auto lambda = []() {
        std::cout << s.x; // OK!
    };
}
```

В этих случаях переменные не захватываются физически: их использование сводится к инлайнингу констант и их фиксированных адресов. Отдельно стоит упомянуть про запрет на наличие `mutable` полей в последнем случае: он связан с тем, что в момент компиляции состояние объекта не будет известно заведомо, хотя он и будет создан.

В общем случае блок `captures` состоит двух подблоков: `capture-default` и `capture-list`. Каждый из них может быть пустым.

`capture-default` развивает идею неявного захвата и позволяет компилятору автоматически захватывать все локальные переменные, которые используются в теле

лямбды. Он имеет ровно две разновидности: *по значению* (`=`) и *по ссылке* (`&`).

```
int main() {
    int x = 3, y = 1, z = 0, w = 4;
    auto lambda = [&]() {
        std::cout << x << y << z << w;
        // если не захватить, будет ошибка компиляции
    };
}
```

`capture-list` позволяет явно перечислить все переменные, которые будут захвачены определяемой лямбдой. Он представляет собой список из разделённых запятой отдельных захватов, которые также бывают двух основных видов: *по значению* (`[identifier]`) и *по ссылке* (`[&identifier]`); отдельно также выделяются захваты `[this]` и `[*this]`, которые позволяют захватить текущий объект по ссылке и по значению соответственно. Поиск имён в списке захвата происходит вплоть до области видимости первой объемлющей функции.

При этом если в блоке `captures` объявлен захват по умолчанию, то все последующие не могут дублировать его семантику, то есть записи `[&, &x]` и `[=, x]` не допустимы; более того, при захвате по умолчанию по значению до C++20 не допускается указание `this` ; в старых стандартах этот указатель захватывался неявно.

Захват также может содержать инициализатор. В этом случае он ведет себя так, как если бы он объявлял и явно захватывал переменную, объявленную со спецификатором `auto` и тем же инициализатором в теле лямбда-выражения, с поправкой на то, что:

- если захват происходит по значению, переменная на самом деле не создаётся, а описанное выше становится верно для нестатического поля самой лямбды (*см. Техническое устройство лямбд*)
- если захват происходит по ссылке, время её жизни есть время жизни лямбды.

Это можно использовать, например, для захвата move-only типов или для захвата по константной ссылке:

```
int main() {
    int x = 238;
    auto lambda = [&cr = std::as_const(x)]() {
        std::cout << cr;
    };
}
```

Лямбда может определить захват по умолчанию или неинициализирующий захват лишь если объемлющая область видимости — это область видимости блока, или если она является частью инициализатора поля класса по умолчанию; причём в этом случае она не может ничего захватывать, если только для каждого захвата не определён инициализатор.

Ещё один интересный аспект, который можно затронуть — это рекурсивные лямбды. До C++23 единственный способ их реализовать выглядел примерно так:

```
auto fact = [](int x) {
    auto impl = [](int x, const auto& impl) {
        if (x == 1) {
            return 1;
        }
        return x * impl(x - 1, impl);
    };
    return impl(5, impl);
};
```

Ужасно ли это? Скорее да.
В C++23 было одобрено предложение P0847R1 — Deducing this — которое не будет подробно затронуто в рамках текущей беседы, но которое позволяет нам реализовать предыдущую лямбду следующим образом:

```
auto fact = [](this auto&& self, int x) {
    if (x == 1) {
        return 1;
    }
    return x * self(x - 1);
};
```

Шаблонные и обобщённые лямбды

В качестве типа параметра в лямбда-выражении можно указывать ключевое слово `auto`. Получившаяся лямбда будет называться *обобщённой* (*generic lambda*):

```
auto Print = [](const auto& x) {
    std::cout << x;
};
```

Для такой лямбды компилятор сгенерирует шаблонный оператор скобок, сопоставив *каждому* `auto` отдельный шаблонный параметр.

Начиная с C++20, лямбда-выражения поддерживают также явное указание шаблонных параметров:

```
auto Print = []<typename T>(const T& x) {
    std::cout << x;
};
```

Такой синтаксис позволяет наложить более строгие ограничения на типы аргументов, а также получить к ним доступ без костылей в виде `decltype`.

Оба этих варианта позволяют реализовывать вариадические лямбды:

```
auto Print = [](auto&&... args) {
    (std::cout << ... << args);
}
```

и, помимо прочего, раскрытие наборов параметров поддерживается и при захвате:

```
template<typename... Args>
void foo(Args&&... args) {
    auto Strange = [&args...]() {
        // something something
    };
}
```

Техническое устройство лямбд

С точки зрения языка лямбда — это объект уникального неименованного (*т.е. практически имеющем уникальное сокрытое имя*) неагрегатного типа, определенного в объемлющем поле видимости объявления и известного как *тип замыкания* (*closure type*).

Абстрактно, этот тип определяет следующие члены:

> ▶ `ClosureType::operator()(<params>)`
>
> Это основной метод, выполняющийся при вызове лямбды. Он исполняет её тело. Важно, что он по умолчанию константный, что значит, что в теле лямбды нельзя просто так изменить значение захваченных переменных:
>
> ```
> auto lambda = [x = 5]() {
>  x = 7; // CE
> }
> ```

> Для того, чтобы убрать константность оператора скобок,[2] используется квалификатор `mutable`:
>
> ```
> auto lambda = [x = 5]() mutable {
>  x = 7; // OK!
> }
> ```
>
> К лямбда-выражению также можно применять квалификаторы `constexpr` (C++17), `consteval` (C++20), `static` (C++23), и каждый из них скажется на сигнатуре оператора скобок.

> ► `ClosureType::operator <ret>(*)(<params>)()`
>
> Как видно из сигнатуры, это оператор преобразования в указатель на функцию. Он определён лишь тогда, когда лямбда не содержит захватов. Внутри типа замыкания генерируется статическая функция, которая выполняет тело лямбды.

> ► Специальные методы
>
> Начиная с C++20, лямбда без захватов может конструироваться по умолчанию, копироваться и перемещаться; если захваты есть (*то есть если блок* `captures` *не пустой; реальных захватов может и не произойти*), соответствующие методы отсутствуют или удалены. Деструктор всегда определяется по умолчанию.

> ► Захваты
>
> Каждая захваченная по значению переменная становится неименованным нестатическим полем класса типа замыкания, определенным при помощи прямой инициализации. Тип соответствующего поля есть тип захваченного объекта, за исключением ссылочных типов: ссылки на объекты захватываются как их копии.
>
> В случае захвата по ссылке Стандарт не обязывает класс создавать отдельное поле.

Что нас здесь интересует особо — это каст к указателю на функцию. Если лямбды не имеют захвата, мы можем создавать целые их коллекции:

```
using FuncPtr = void(*)();
std::vector<FuncPtr> f_col;
f_col.push_back([](){ std::cout << "I'm a lambda!"; });
```

Это — да и не только — очень здорово и удобно и так далее, но.
Когда мы начинаем смотреть на нечто очень сильно подобное функции (настолько подобное, что аж полностью обратно совместимое с указателем на неё) как на объект, мы

неизбежно сталкиваемся с необходимостью закрыть один давний пробел, что в итоге приведет нас к очень важной мысли о природе вызываемых объектов вообще.

Полиморфизм вызываемых объектов

А что такое функция?

Действительно: мы уже не раз заводили псевдоним для указателя на функцию, но ни разу не задумывались о том, что будет, если его разыменовать. Иначе говоря, что есть функция с точки зрения системы типов языка?

Мы заведомо знаем, что функция не является объектом: мы не можем создать их коллекцию или передать одну функцию в другую непосредственно. Однако совершенно никто не постулировал, что тип — нечто присущее лишь объекту; вспомнить хотя бы `void` или абстрактные классы. В статически типизированных языках тип выступает в первую очередь контрактом между сущностью и компилятором, и для функций это актуально как никогда.

Функциональный тип существует и, в первую очередь, является неполным, что строго закрепляет необъектность функций. Он несет в себе информацию о возвращаемом значении, типах каждого из аргументов, о том, является ли функция `noexcept`, и, в случае нестатических методов класса, о квалификаторах и ссылочности.

Функциональный тип предпочитается указателю тогда, когда нам необходима полная информация о сигнатуре. Это полезно в первую очередь в шаблонах и их специализациях; вот так, например, может выглядить трейт, определяющий, можно ли вызвать функцию с некоторым набором аргументов:

```
template<typename T, typename Signature>
struct IsCallableWith : std::false_type {};

template<typename T, typename R, typename... Args>
struct IsCallableWith<T, R(Args...)> {
     template<typename U>
     static auto Check(int) -> decltype(
         std::declval<U>()(std::declval<Args>()...),
        std::true_type {}
     );

     template<typename U>
     static auto Check(...) -> std::false_type {};

     using value = decltype(test<T>(0));
};
```

Код выше — это, впрочем, скорее развлечение; куда более интересная мысль состоит в том, что теперь, когда мы знаем о сигнатуре функции практически всё, мы можем зайти довести нашу эпопею до логического конца и попытаться создать полиморфную обёртку, которая могла бы хранить и вызывать как функцию, так и лямбду или даже функтор.

`std::function`

Проблема в том, что функторы и функции столь же разнородны, сколько похожи, в связи с чем выделить общую их семантику в единственный класс — задача не совсем тривиальная. Мы не можем просто сохранить вложенный объект как поле, ибо тип его нам заведомо неизвестен.

Хотя, вообще говоря, мы уже умеем решать такую задачу. Что мы обычно делаем, когда нам нужно оставить от типа лишь фиксированный набор методов, вынеся при этом их реализацию в какое-то другое место? Мы создаём интерфейс.

В этом месте возникает очень мощная и важная идиома — **Type Erasure** — которая позволяет буквально спрятать конкретный тип объекта за интерфейсным классом. Идея проста:

1. есть базовый абстрактный класс, по которому внешняя программа будет обращаться к объекту со стёртым типом;
2. есть *шаблонный* наследник, который представляет собой конкретный объект с известным типом;
3. есть внешний класс-обёртка, содержащий *указатель или ссылку на базу*, по которому хранится наследник.

Говоря более общими терминами, мы используем *параметрический полиморфизм*, чтобы объединить несколько несвязанных между собой объектов в одно семейство, и *полиморфизм подтипов*, чтобы получать доступ к этим объектам при помощи универсального интерфейса.

Выглядит это всё примерно следующим образом:

```
template<typename T>
class Function {};

template<typename R, typename... Args>
class Function<R(Args...)> {
  public:
    class ICallable {
      public:
        virtual R operator()(Args... args) const = 0;
        virtual ~ICallable() = default;
```

```
    };

    template<typename T>
    class CallableImpl : public ICallable {
      public:
        CallableImpl(const T& callable)
            : callable_(callable)
        {}

        R operator()(Args... args) const override {
            return callable_(args...);
        }

      private:
        T callable_;
    };

    template<typename T>
    Function(T&& callable)
        : callable_(std::make_unique<CallableImpl<T>>(CallableImpl<T>
(std::forward<T>(callable))))
    {}

    R operator()(Args... args) const {
        return callable_->operator()(args...);
    }

  private:
    std::unique_ptr<ICallable> callable_;
};
```

Ровно эта идея используется в основе стандартного класса `std::function`, который мы уже мельком затрагивали ранее. Он позволяет хранить, копировать и вызвать любой вызываемый объект. Если объект типа `std::function` не содержит никакого целевого объекта, то его вызов приведёт к исключению `std::bad_function_call`.

Источники

1. А. П. Хвастунов — Лекции по основам программирования на C++, 2 семестр, 2026
2. cppreference.com — Lambda expressions (https://en.cppreference.com/w/cpp/language/lambda.html)
3. Wikipedia — Замыкание (программирование) (https://ru.wikipedia.org/wiki/

Замыкание_(программирование))
4. Proposal P0847R1 — Deducing this (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0847r1.html)
5. cppreference.com — Functions (https://en.cppreference.com/w/cpp/language/functions.html)

---

1. что, вообще-то, забавно: вывод возвращаемого типа формально появился лишь в C++14, тогда как лямбды — в C++11, но этот механизм всё равно работал даже тогда. ↩
2. при помощи cppinsights можно убедиться, что это действительно весь эффект, который оказывает квалификатор mutable. ↩